



Class Notes: Understanding Cursors

What is a Cursor?

A cursor is a temporary work area where SQL stores the result set of a query. It allows you to iterate over the result set one row at a time, performing operations on each row.

Why Use Cursors?

Cursors are useful when you need to:

1. Process data row by row.
2. Perform complex operations that can't be done in a single SQL statement.
3. Iterate over a result set and perform actions based on the data.

How Cursors Work

Here's a step-by-step overview of how cursors work:

1. *Declare the cursor*: You define a cursor with a SELECT statement that retrieves the data you want to process.
2. *Open the cursor*: You open the cursor, which prepares it for use.
3. *Fetch rows*: You fetch rows from the cursor one at a time, storing the values in variables.
4. *Process the data*: You perform operations on the data in the variables.
5. *Close and deallocate*: Finally, you close and deallocate the cursor to free up resources.

Example: Using a Cursor to Print Employee Names

Suppose we have a table called `Employees` with columns `EmployeeID` and `Name`. We want to print each employee's name using a cursor.

```

-- Declare variables
DECLARE @EmployeeID INT
DECLARE @Name NVARCHAR(50)

-- Declare the cursor
DECLARE employee_cursor CURSOR FOR
SELECT EmployeeID, Name
FROM Employees

-- Open the cursor
OPEN employee_cursor

-- Fetch the first row
FETCH NEXT FROM employee_cursor INTO @EmployeeID, @Name

-- Loop through the rows
WHILE @@FETCH_STATUS = 0
BEGIN
    PRINT 'Employee ID: ' + CAST(@EmployeeID AS NVARCHAR) + ', Name: ' + @Name
    FETCH NEXT FROM employee_cursor INTO @EmployeeID, @Name
END

-- Close and deallocate the cursor
CLOSE employee_cursor
DEALLOCATE employee_cursor

```

In this example:

1. We declare variables to store the employee ID and name.
2. We declare a cursor that retrieves the employee ID and name from the `Employees` table.
3. We open the cursor and fetch the first row.
4. We loop through the rows, printing each employee's name.
5. Finally, we close and deallocate the cursor.

Key Concepts

- `@@FETCH_STATUS`: This variable returns the status of the last fetch operation. A value of 0 indicates success.
- `FETCH NEXT`: This statement retrieves the next row from the cursor.

- *CLOSE*: This statement closes the cursor.
- *DEALLOCATE*: This statement removes the cursor definition and frees up resources.

By understanding cursors, you'll be able to write more complex SQL code that can handle row-by-row operations.